Rapport Cloud Computing

Mélissa Emorine

Sommaire

Initialisation du Terraform....2
Configuration du fichier main.tf....2
Configuration du fichier variables.tf....3
Configuration du fichier output.tf....3
Déploiement de l’infrastructure Terraform....4
Lancement de la commande terraform init....4
Lancement de la commande terraform plan....5
Lancement de la commande terraform apply....5
Mise en place du Blob Storage....9
Configuration Terraform....9
Déploiement et résolution des erreurs....9
Mise en place du backend....12
Déploiement avec Ansible....13
Préparation de la VM....13
Shell Script pour déployer....13
Variables d’environnement....14
Autoriser l’API dans Terraform....14
Organisation du projet et configuration Ansible....14
Mise en place de l'inventaire....14
Mise en place du playbook....15
Erreurs de permission....15
Initialisation de la Base de Données SQL....16
Configuration de la base de données dans Terraform....16
Déploiement et résolution des erreurs....16
Avertissements de dépréciation Ansible....17
Provisionnement de la Base de Données....18
Échec du déploiement Ansible....18
Erreur 404 lors du terraform apply....19
Suppression de la base de données....20
Conclusion....22

Initialisation du Terraform

La mise en place de l'infrastructure a débuté par la création d'un répertoire dédié à Terraform, dans lequel les trois fichiers de configuration standards ont été initialisés : *main.tf*, *variables.tf* et *outputs.tf*.

```
000 tp_4_cloud_computing $ mkdir terraform
000 tp_4_cloud_computing $ mkdir project
000 tp_4_cloud_computing $ touch README.md
000 tp_4_cloud_computing $ cd terraform/
000 terraform $ touch main.tf
000 terraform $ touch variables.tf
000 terraform $ touch outputs.tf
```

L'authentification auprès d'Azure a ensuite été effectuée via la commande az login, permettant d'établir une session active avec le compte Azure cible avant tout déploiement.

Configuration du fichier main.tf

La mise en place du fichier *main.tf* a débuté par la déclaration des éléments requis pour l'initialisation de Terraform, à savoir la configuration du bloc terraform ainsi que la définition du provider Azure.

Afin d'accélérer le déploiement, un exemple de configuration de machine virtuelle issu de la documentation officielle a servi de base de travail. Plusieurs adaptations ont ensuite été apportées pour répondre aux exigences du projet :

- **Région de déploiement** : la propriété location a été définie sur France Central conformément aux contraintes géographiques du projet.
- **Dimensionnement de la VM** : la propriété *vm_size* a été configurée sur *Standard_D2s_v3* afin de correspondre aux ressources nécessaires.
- **Image système** : l'image OS utilisée dans l'exemple de référence, bien que toujours supportée, n'était plus à jour. La commande az vm image list a été utilisée pour identifier une image récente. La commande az vm image list --all --publisher Canonical a également été testée pour lister l'ensemble des images Canonical, mais son exécution s'est avérée trop longue en raison du volume de résultats retournés. Une image Ubuntu Noble a finalement été sélectionnée.
- **Politique de suppression des disques** : les options *delete_os_disk_on_termination* et *delete_data_disks_on_termination* ont été activées afin de garantir une destruction propre et complète des ressources lors d'un terraform destroy.
- **Adresse IP publique** : une ressource d'IP publique a été déclarée et associée à l'interface réseau de la VM.
- **Accès SSH** : le bloc *os_profile_linux_config* a été configuré pour autoriser l'authentification par clé SSH, et une ressource de clé publique SSH a été ajoutée à la configuration.

Configuration du fichier variables.tf

Les informations sensibles liées au profil OS, notamment les identifiants de connexion, ont été extraites du fichier principal et centralisées dans le fichier *variables.tf* afin d'éviter l'exposition de secrets.

Configuration du fichier output.tf

Un output dédié à l'adresse IP publique de la VM a été défini dans le fichier *outputs.tf*, permettant de récupérer facilement cette valeur à l'issue d'un terraform apply.

Déploiement de l’infrastructure Terraform

Après cette première configuration, il était temps de lancer Terraform afin de vérifier son fonctionnement.

Lancement de la commande terraform init

La première exécution a retourné une erreur due à une accolade manquante dans la configuration.

```
000 terraform $ terraform init
Initializing the backend...

  Error: Terraform encountered problems during initialisation, including problems
  with the configuration, described below.

  The Terraform configuration must be valid before initialization so that
  Terraform can determine which modules and providers need to be installed.



  Error: Unclosed configuration block

    on variables.tf line 6, in variable "computer_name":
     6: variable "computer_name" {

  There is no closing brace for this block before the end of the file. This may be caused by incorrect
  brace nesting elsewhere in this file.
```

Après correction, l'initialisation s'est déroulée sans encombre.

```
000 terraform $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Lancement de la commande terraform plan

Cette étape s’est exécutée sans erreur.

```
Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ip_publique = (known after apply)
```

Lancement de la commande terraform apply

Plusieurs erreurs sont survenues avant d'obtenir un déploiement fonctionnel.

La première erreur concernait l'admin_username : le nom d'utilisateur « admin » est explicitement interdit par Azure, car il s'agit d'un nom réservé présentant des risques de sécurité.

```
Error: creating/updating Virtual Machine (Subscription: "87b59e8c-1393-4d4a-b46c-8b645a549d04"
Resource Group Name: "caca-resources"
Virtual Machine Name: "caca-vm"): performing CreateOrUpdate: unexpected status 400 (400 Bad Request) w
ith error: InvalidParameter: The Admin Username specified is not allowed. For more information about dis
allowed usernames, see https://aka.ms/vmosprofile

  with azurerm_virtual_machine.main,
  on main.tf line 61, in resource "azurerm_virtual_machine" "main":
  61: resource "azurerm_virtual_machine" "main" {
```

La variable a été corrigée dans le fichier *variables.tf* et la commande terraform apply relancée.

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

ip_publique = "20.19.250.243"
```

La vérification sur Azure montre que les différentes ressources ont bien été créées.

Name ↑	Type	Location
caca-network	Virtual network	France Central
caca-nic	Network Interface	France Central
caca-ssh	SSH key	France Central
caca-vm	Virtual machine	France Central
ip-public	Public IP address	France Central
myosdisk1	Disk	France Central

La connexion SSH, quant à elle, ne fonctionnait pas : bien que la VM ait été correctement provisionnée et visible dans le portail Azure, la tentative de connexion SSH restait bloquée au handshake sur le port 22. Après vérification, le port SSH n'était pas listé dans les ports de la machine créée.

```
130 terraform $ nmap 20.19.250.243
Starting Nmap 7.92 ( https://nmap.org ) at 2026-03-12 10:59 CET
Nmap scan report for 20.19.250.243
Host is up (0.15s latency).
Not shown: 920 filtered tcp ports (no-response), 75 filtered tcp ports (host-unreach)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
443/tcp  open  https
554/tcp  open  rtsp
1723/tcp open  pptp
```

L'origine du problème résidait dans l'absence d'un Network Security Group, indispensable pour autoriser le trafic entrant sur ce port.

J'ai récupéré mon adresse IP et l'ai mise dans une variable afin de n'autoriser que les connections via mon adresse IP publique.

Afin de restreindre l'accès SSH à la seule machine de travail, l'adresse IP publique a été récupérée via la commande suivante : curl https://api.ipify.org Cette adresse a ensuite été intégrée dans une variable et utilisée comme règle d'entrée dans le Network Security Group, limitant ainsi les connexions SSH à cette unique source.

À l'issue de cette correction, la connexion SSH a été établie avec succès.

```
130 terraform $ ssh -i ~/.ssh/azure_key testadmin@20.19.250.243
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1017-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Mar 12 10:20:04 UTC 2026

  System load:  0.08               Processes:             121
  Usage of /:   5.7% of 28.02GB    Users logged in:       0
  Memory usage: 3%                 IPv4 address for eth0: 10.0.2.4
  Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Mar 12 10:20:07 2026 from 87.88.153.204
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

testadmin@computer:~$
```

Mise en place du Blob Storage

Configuration Terraform

Afin de permettre le stockage de fichiers statiques tels que des images et des logs, un Blob Storage Azure a été ajouté à l'infrastructure existante. La configuration s'appuie sur plusieurs ressources Terraform : un compte de stockage *azurerm_storage_account*, deux containers distincts images et logs, ainsi que des règles d'accès via *azurerm_role_assignment*.

Dans une optique de sécurisation dès la conception, plusieurs mesures ont été appliquées directement dans la configuration : désactivation de l'accès public anonyme, forçage du protocole HTTPS et mise en place d'un pare-feu réseau restreignant les accès au seul subnet de la VM et à l'adresse IP de la machine de travail.

Une politique de cycle de vie a également été configurée sur le container de logs, afin de maîtriser les coûts de stockage : les fichiers sont automatiquement basculés en tier froid après 30 jours, archivés après 90 jours, puis supprimés au bout d'un an.

Déploiement et résolution des erreurs

Le déploiement a nécessité plusieurs itérations avant d'aboutir.

Problème de timing sur le principal_id

```
Error: Missing required argument

  with azurerm_role_assignment.vm_blob_contributor,
  on main.tf line 221, in resource "azurerm_role_assignment" "vm_blob_contributor":
 221:   principal_id         = azurerm_virtual_machine.main.identity[0].principal_id

The argument "principal_id" is required, but no definition was found.
```

La première difficulté est survenue lors de l'attribution des permissions administrateur sur le Storage Account. Le bloc *azurerm_role_assignment suivant* est chargé de donner les droits Storage Blob Data Owner à l'identité courante (celle déclarée dans la VM) :

```
resource "azurerm_role_assignment" "vm_blob_contributor" {
  scope                = azurerm_storage_account.main.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_virtual_machine.main.identity[0].principal_id
  depends_on           = [azurerm_virtual_machine.main]
}
```

L'argument principal_id pointe ici vers *data.azurerm_client_config.current.object_id*, dont le rôle est de récupérer dynamiquement l'identité de l'administrateur qui exécute Terraform. Or, au moment où Terraform résout ce bloc, la ressource data *"azurerm_client_config" "current" {}* est encore vide : l'identité managée n'est pas encore instanciée.

Dans un premier temps, une tentative de réorganisation du code a été effectuée, en remontant le bloc identity plus tôt dans la déclaration de la VM, sans succès — Terraform ne résout pas les dépendances en fonction de l'ordre d'écriture dans le fichier.

La solution a consisté à procéder en deux passes distinctes. La première exécution, ciblée via terraform apply -target=azurerm_virtual_machine.main, permet de créer uniquement la VM et d'instancier son identité managée. Une fois cette étape terminée, le terraform plan complet a été relancé : l'erreur ne s'est plus manifestée. Il s'agissait donc bien d'un problème de timing, l'identité devant exister dans Azure avant de pouvoir être référencée par les ressources qui en dépendent.

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

ip_publique = "20.19.250.243"
storage_account_name = "cacastorage"
storage_primary_access_key = <sensitive>
storage_primary_endpoint = "https://cacastorage.blob.core.windows.net/"
```

Erreur NetworkAclsValidationFailure

Une seconde erreur est apparue lors de la création du Storage Account. Azure a retourné une erreur *NetworkAclsValidationFailure*, indiquant que le subnet interne ne disposait pas du Service Endpoint *Microsoft.Storage*.

```
  Resource Group Name: "caca-resources"
  Storage Account Name: "cacastorage"): polling after Create: polling failed: the Azure API returned the
 following error:

  Status: "NetworkAclsValidationFailure"
  Code: ""
  Message: "Validation of network acls failure: SubnetsHaveNoServiceEndpointsConfigured:Subnets internal
 of virtual network /subscriptions/87b59e8c-1393-4d4a-b46c-8b645a549d04/resourceGroups/caca-resources/pr
oviders/Microsoft.Network/virtualNetworks/caca-network do not have ServiceEndpoints for Microsoft.Storag
e resources configured. Add Microsoft.Storage to subnet's ServiceEndpoints collection before trying to A
CL Microsoft.Storage resources to these subnets.."
  Activity Id: ""
```

En effet, lorsqu'un subnet est référencé dans les règles réseau d'un Storage Account, Azure exige qu'un Service Endpoint soit explicitement activé sur ce subnet afin d'établir un chemin réseau direct et sécurisé vers le service de stockage, sans transit par Internet. La correction a consisté à ajouter service_endpoints = ["Microsoft.Storage"] sur la ressource *azurerm_subnet*.

```
resource "azurerm_subnet" "internal" {
  name                 = "internal"
  resource_group_name  = azurerm_resource_group.example.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.2.0/24"]
  service_endpoints    = ["Microsoft.Storage"]
}
```

Dépréciations identifiées

Une revue de la configuration a par ailleurs permis d'identifier et de corriger un éléments qui deviendra déprécié en version 4.x : l'argument *enable_https_traffic_only* initialement instancié a été renommé en *https_traffic_only_enabled*.

À l'issue de ces corrections, le déploiement s'est finalisé avec succès, et le Blob Storage est opérationnel avec les deux containers et les permissions attendues.

```
Apply complete! Resources: 5 added, 1 changed, 0 destroyed.

Outputs:

ip_publique = "20.19.250.243"
storage_account_name = "cacastorage"
storage_primary_access_key = <sensitive>
storage_primary_endpoint = "https://cacastorage.blob.core.windows.net/"
```

Mise en place du backend

L'application backend repose sur une API Flask minimaliste exposant deux routes. La mise en place a débuté par la création de l'arborescence de dossiers nécessaire, suivie de l'initialisation d'un environnement virtuel Python afin d'isoler les dépendances du projet.

```
000 tp_4_cloud_computing $ cd project/
000 project $ touch app.py
000 project $ touch requirements.txt
000 project $ vim app.py
000 project $ vim requirements.txt
000 project $ python -m venv .venv
```

```
002 project $ source .venv/bin/activate
(.venv) 000 project $ pip install -r requirements.txt
Collecting flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask->-r requirements.txt (line 1))
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask->-r requirements.txt (line 1))
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
```

Des tests locaux ont été effectués pour valider le bon fonctionnement de l'API avant tout déploiement. L'ensemble des routes a répondu correctement.

```
(.venv) 000 project $ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI s
erver instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 389-545-955
127.0.0.1 - - [12/Mar/2026 14:02:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Mar/2026 14:02:15] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Mar/2026 14:02:23] "GET /services HTTP/1.1" 200 -
127.0.0.1 - - [12/Mar/2026 14:02:35] "GET / HTTP/1.1" 200 -
```

Suite à des modifications apportées à la configuration Terraform, un terraform apply a été lancé afin de synchroniser l'infrastructure avec l'état souhaité.

Déploiement avec Ansible

Préparation de la VM

Afin de préparer à l'installation de dépendances sur la machine, un apt update suivi d'un apt upgrade a été lancé sur la machine.

```
testadmin@computer:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
```

```
testadmin@computer:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following NEW packages will be installed:
Installing new version of config file /etc/lsb-release ...
motd-news.service is a disabled or a static unit not running, not starting it.
(Reading database ... 68452 files and directories currently installed.)
Preparing to unpack .../libc-bin_2.39-0ubuntu8.7_amd64.deb ...
Unpacking libc-bin (2.39-0ubuntu8.7) over (2.39-0ubuntu8.6) ...
Setting up libc-bin (2.39-0ubuntu8.7) ...
(Reading database ... 68452 files and directories currently installed.)
Preparing to unpack .../0-libexpat1_2.6.1-2ubuntu0.4_amd64.deb ...
Unpacking libexpat1:amd64 (2.6.1-2ubuntu0.4) over (2.6.1-2ubuntu0.3) ...
Preparing to unpack .../1-libpython3.12t64_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking libpython3.12t64:amd64 (3.12.3-1ubuntu0.12) over (3.12.3-1ubuntu0.10) ...
Preparing to unpack .../2-python3.12_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking python3.12 (3.12.3-1ubuntu0.12) over (3.12.3-1ubuntu0.10) ...
Preparing to unpack .../3-libpython3.12-stdlib_3.12.3-1ubuntu0.12_amd64.deb ...
Unpacking libpython3.12-stdlib:amd64 (3.12.3-1ubuntu0.12) over (3.12.3-1ubuntu0.10) ..

Progress: [ 13%] [#########################.........................................
```

Shell Script pour déployer

Afin de permettre le déploiement et la mise en place du serveur en même temps, il a été décidé de réaliser un script qui charge les données nécessaires au déploiement, exécute les commandes terraform nécessaires puis effectue le déploiement avec Ansible.

Variables d’environnement

Les variables sensibles et partagées entre Ansible et Terraform sont centralisées dans un fichier *.env* à la racine du projet. Le script *deploy.sh* les charge au démarrage, les exporte ensuite vers Terraform sous forme de variables *TF_VAR_**. Ansible récupère les données dans le *.env* grâce à lookup, une extension du Jinja2 template. Cette approche permet de n'avoir qu'une seule source de vérité pour l'ensemble de la configuration.

Autoriser l’API dans Terraform

Pour rendre l'API accessible depuis l'extérieur, une règle de sécurité réseau supplémentaire a été ajoutée dans Terraform, en suivant le même modèle que la règle SSH définie précédemment, en adaptant le nom, la priorité et le numéro de port.

```
security_rule {
  name                       = "allow-api"
  priority                   = 110
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "Tcp"
  source_port_range          = "*"
  destination_port_range     = "8000"
  source_address_prefix      = "${var.my_ip_address}/32"
  destination_address_prefix = "*"
}
```

Le terraform apply s'est effectué avec succès.

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

ip_publique = "20.19.250.243"
storage_account_name = "cacastorage"
storage_primary_access_key = <sensitive>
storage_primary_endpoint = "https://cacastorage.blob.core.windows.net/"
```

Organisation du projet et configuration Ansible

Le projet Ansible s'articule autour de trois composants principaux : un fichier *playbook.yml* qui décrit l'ensemble des tâches à exécuter, un fichier *inventory.ini* qui référence les machines cibles (ici une seule), et des fichiers de variables qui centralisent la configuration. Les fichiers de variables ont dû être séparés en 2 instances : *vars.yml* qui charge les données lorsqu’Ansible est déjà lancé, et *group_vars/* qui charge les données au lancement d’Ansible, afin d’éviter l’absence de certaines variables nécessaires au lancement.

Mise en place de l'inventaire

Le fichier *inventory.ini* déclare la VM Azure, dont les paramètres de connexion réels sont résolus dynamiquement via les variables définies dans *group_vars*.

Mise en place du playbook

Le playbook orchestre l'ensemble du déploiement de l'API sur la VM. Il prend en charge la mise à jour du système, l'installation des dépendances système, la création d'un utilisateur applicatif dédié, la synchronisation des fichiers du projet, la création de l'environnement virtuel Python, l'installation des dépendances Python, ainsi que la création et l'activation d'un service *systemd* assurant le démarrage automatique de l'API.

Erreurs de permission

Lors du déploiement Ansible, à l'étape de la création de l'environnement Python, des problèmes.

```
TASK [api : Create Python virtual environment] *******************************************************
[ERROR]: Task failed: Failed to set permissions on the temporary files Ansible needs to create when beco
ming an unprivileged user (rc: 1, err: chmod: invalid mode: 'A+user:apiuser:rx:allow'
Try 'chmod --help' for more information.
}). For information on working around this, see https://docs.ansible.com/ansible-core/2.20/playbook_guid
e/playbooks_privilege_escalation.html#risks-of-becoming-an-unprivileged-user
Origin: /home/melordi/Documents/Cours/B3-Web-Dev/14. Cloud Computing/tp_4_cloud_computing/ansible/roles/
api/tasks/main.yml:53:3

51     recurse: yes
52
53 - name: Create Python virtual environment
     ^ column 3

fatal: [azure_vm]: FAILED! => {"changed": false, "msg": "Task failed: Failed to set permissions on the t
emporary files Ansible needs to create when becoming an unprivileged user (rc: 1, err: chmod: invalid mo
de: 'A+user:apiuser:rx:allow'\nTry 'chmod --help' for more information.\n}). For information on working
around this, see https://docs.ansible.com/ansible-core/2.20/playbook_guide/playbooks_privilege_escalatio
n.html#risks-of-becoming-an-unprivileged-user"}

PLAY RECAP *********************************************************************************************
azure_vm                   : ok=7    changed=2    unreachable=0    failed=1    skipped=0    rescued=0
 ignored=0
```

La solution a consisté à créer un fichier *ansible.cfg*.

```
PLAY RECAP *********************************************************************************************
azure_vm                   : ok=14   changed=7    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

API accessible sur : http://20.19.250.243:8000
```

Avec ce changement, le déploiement se déroule avec succès et l'API est bien disponible à l'adresse indiquée.

Initialisation de la Base de Données SQL

Configuration de la base de données dans Terraform

La configuration s'est appuyée sur l'exemple fourni par la documentation officielle de Microsoft. L'analyse du code de référence a mis en évidence l'utilisation d'une ressource *pet name*, dont le rôle est de générer des noms aléatoires. Ce mécanisme a été remplacé par le provider *random* de Terraform, dont la version la plus récente a été consultée via la documentation officielle. Il a été utilisé pour générer une chaîne de dix caractères aléatoires, appliquée à la fois comme nom de la base de données et comme suffixe du compte de stockage, ce dernier nécessitant également un identifiant unique.

Le code a ensuite été épuré pour ne conserver que les blocs strictement nécessaires : *azurerm_mysql_server* et *azurerm_mysql_database*. Les valeurs ont été adaptées pour correspondre aux variables d'environnement et aux ressources déjà déclarées dans le projet.

Déploiement et résolution des erreurs

Le déploiement a été lancé via la commande `./ansible/deploy.sh -vvv`. Avant de pouvoir réellement déployer, deux itérations ont été nécessaires à cause d'une erreur de complexité du mot de passe de la base de données.

```
  Error: creating Server (Subscription: "87b59e8c-1393-4d4a-b46c-8b645a549d04"
  Resource Group Name: "caca-resources"
  Server Name: "caca-sql-4dv5mj5ol1"): polling after CreateOrUpdate: polling failed: the Azure API retur
ned the following error:

  Status: "PasswordNotComplex"
  Code: ""
  Message: "Password validation failed. The password does not meet policy requirements because it is not
 complex enough."
  Activity Id: ""

  ---

  API Response:
```

En effet, le mot de passe initialement utilisé était identique à celui de la VM, or Azure SQL impose des exigences de complexité distinctes. Afin de s'aligner avec les bonnes pratiques de sécurité, de nouveaux identifiants dédiés ont été créés dans le fichier *.env.*

```
# Database
DB_NAME=              # Nom de la base de données
DB_USER=              # Nom d'utilisateur pour la base de données
DB_PASSWORD=          # Mot de passe pour la base de données
```

Après avoir relancé le déploiement, l'erreur persistait.

```
  Resource Group Name: "caca-resources"
  Server Name: "caca-sql-4dv5mj5ol1"): polling after CreateOrUpdate: polling failed: the Azure API retur
ned the following error:

  Status: "PasswordNotComplex"
  Code: ""
  Message: "Password validation failed. The password does not meet policy requirements because it is not
 complex enough."
  Activity Id: ""
```

D'après la politique de mots de passe SQL Server, le mot de passe doit contenir au moins trois des quatre catégories suivantes : majuscules, minuscules, chiffres et caractères spéciaux. Ces variables ont été déclarées dans *variables.tf*, référencées dans *main.tf*, et ajoutées aux variables requises dans *deploy.sh* pour assurer leur injection correcte. Après ajout d'un chiffre supplémentaire au mot de passe, le déploiement a abouti avec succès.

```
PLAY RECAP *********************************************************************************************
azure_vm                   : ok=13   changed=4    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

API accessible sur : http://20.19.250.243:8000
```

Avertissements de dépréciation Ansible

À l'issue du déploiement, plusieurs avertissements ont été relevés, liés à l'utilisation de modules Ansible dépréciés :

```
TASK [api : Create application directory] ***************************************************************
ok: [azure_vm] => {"changed": false, "gid": 988, "group": "apiuser", "mode": "0755", "owner": "apiuser",
 "path": "/opt/api", "size": 4096, "state": "directory", "uid": 999}
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: Importing 'to_text' from 'ansible.module_utils._text' is deprecated. This feature
 will be removed from ansible-core version 2.24. Use ansible.module_utils.common.text.converters instead
.
[DEPRECATION WARNING]: The `ansible.module_utils.common._collections_compat` module is deprecated. This
feature will be removed from ansible-core version 2.24. Use `collections.abc` from the Python standard l
ibrary instead.
```

```
TASK [api : Copy application files to VM] ***************************************************************
[DEPRECATION WARNING]: Importing 'to_bytes' from 'ansible.module_utils._text' is deprecated. This featur
e will be removed from ansible-core version 2.24. Use ansible.module_utils.common.text.converters instea
d.
[DEPRECATION WARNING]: Importing 'to_native' from 'ansible.module_utils._text' is deprecated. This featu
re will be removed from ansible-core version 2.24. Use ansible.module_utils.common.text.converters inste
ad.
```

Ces avertissements n'ont pas bloqué le déploiement mais indiquent que certaines dépendances internes d'Ansible utilisées sont amenées à être supprimées.

Provisionnement de la Base de Données

La connexion à la base de données est assurée via le client Python *pymssql*, dont la dépendance a été ajoutée au fichier *requirements.txt*.

La structure de la base de données a été initialisée directement depuis l'application : la création de la table est effectuée en premier lieu dans le point d'entrée main, garantissant ainsi que le schéma est en place avant tout traitement des requêtes.

Plusieurs ajustements ont été apportés à la configuration pour permettre la communication entre les différentes couches de l'infrastructure.

Une règle firewall Terraform a été ajoutée dans *main.tf* pour autoriser la VM à accéder au serveur SQL.

Le nom de domaine complet (FQDN) du serveur SQL a été exposé dans *output.tf* afin de le rendre accessible aux étapes suivantes du pipeline de déploiement afin de ne pas coder en dur une valeur susceptible de changer selon l'environnement.

Le script de déploiement récupère le FQDN depuis les outputs Terraform et le transmet à Ansible sous forme de variable. Cela permet à Ansible de configurer dynamiquement l'application sans connaissance préalable de l'adresse du serveur.

Les variables de connexion à la base de données (hôte, identifiants, nom de la base) sont injectées dans la définition du service systemd dans le *playbook.yml*, ce qui permet à l'application de les lire depuis son environnement d'exécution.

Échec du déploiement Ansible

À l'issue du déploiement, Ansible met en lumière un problème de déploiement : l'API ne répond pas.

```
TASK [Wait for API to be reachable] ********************************************************************
[ERROR]: Task failed: Module failed: Timeout when waiting for 0.0.0.0:8000
Origin: /home/melordi/Documents/Cours/B3-Web-Dev/14. Cloud Computing/tp_4_cloud_computing/ansible/playbo
ok.yml:126:7

124         daemon_reload: yes
125
126     - name: Wait for API to be reachable
          ^ column 7

fatal: [azure_vm]: FAILED! => {"changed": false, "elapsed": 30, "msg": "Timeout when waiting for 0.0.0.0
:8000"}

PLAY RECAP *********************************************************************************************
azure_vm                   : ok=11   changed=5    unreachable=0    failed=1    skipped=0    rescued=0
 ignored=0
```

La vérification du statut du service api-backend a révélé un état d'échec.

```
testadmin@computer:~$ sudo systemctl status api-backend
● api-backend.service - api-backend - Python API Backend
     Loaded: loaded (/etc/systemd/system/api-backend.service; enabled; preset: enabled)
     Active: activating (auto-restart) (Result: exit-code) since Thu 2026-03-12 20:05:58 UTC; 3s ago
    Process: 26738 ExecStart=/opt/api/.venv/bin/python app.py (code=exited, status=1/FAILURE)
   Main PID: 26738 (code=exited, status=1/FAILURE)
        CPU: 252ms
```

L'analyse des logs via *journalctl* a indiqué une erreur d'authentification à la base de données.

```
systemd[1]: api-backend.service: Scheduled restart job, restart counter is at 56.
systemd[1]: Started api-backend.service - api-backend - Python API Backend.
python[26703]: Traceback (most recent call last):
python[26703]:   File "src/pymssql/_pymssql.pyx", line 651, in pymssql._pymssql.connect
python[26703]:   File "src/pymssql/_mssql.pyx", line 2189, in pymssql._mssql.connect
python[26703]:   File "src/pymssql/_mssql.pyx", line 722, in pymssql._mssql.MSSQLConnection.__init__
python[26703]:   File "src/pymssql/_mssql.pyx", line 1915, in pymssql._mssql.maybe_raise_MSSQLDatabaseException
python[26703]:   File "src/pymssql/_mssql.pyx", line 1932, in pymssql._mssql.raise_MSSQLDatabaseException
python[26703]: pymssql._mssql.MSSQLDatabaseException: (18456, b"Login failed for user 'dbuser'.DB-Lib error mess
python[26703]: During handling of the above exception, another exception occurred:
python[26703]: Traceback (most recent call last):
python[26703]:   File "/opt/api/app.py", line 34, in <module>
python[26703]:     init_db()
python[26703]:   File "/opt/api/app.py", line 8, in init_db
python[26703]:     conn = pymssql.connect(
python[26703]:            ^^^^^^^^^^^^^^^^
python[26703]:   File "src/pymssql/_pymssql.pyx", line 660, in pymssql._pymssql.connect
python[26703]: pymssql.exceptions.OperationalError: (18456, b"Login failed for user 'dbuser'.DB-Lib error messag
systemd[1]: api-backend.service: Main process exited, code=exited, status=1/FAILURE
systemd[1]: api-backend.service: Failed with result 'exit-code'.
```

L'origine du problème a été identifiée dans le fichier *main.tf* : la propriété administrator_login référençait par erreur *var.db_name* au lieu de *var.db_user.* Après correction de cette variable, le déploiement a été relancé.

Erreur 404 lors du terraform apply

Un nouvel obstacle est apparu lors du terraform apply suivant, avec l'erreur :

```
  Resource Group Name: "caca-resources"
  Server Name: "caca-sql-4dv5mj5ol1"): performing CreateOrUpdate: unexpected status 404 (404 Not Found)
with error: ParentResourceNotFound: Failed to perform 'write' on resource(s) of type 'servers/connection
Policies', because the parent resource '/subscriptions/87b59e8c-1393-4d4a-b46c-8b645a549d04/resourceGrou
ps/caca-resources/providers/Microsoft.Sql/servers/caca-sql-4dv5mj5ol1' could not be found.

    with azurerm_mssql_server.server,
    on main.tf line 252, in resource "azurerm_mssql_server" "server":
   252: resource "azurerm_mssql_server" "server" {
```

L'inspection de l'état Terraform a révélé qu'une ressource était marquée comme *tainted*. Conformément à la documentation Terraform, cet état indique qu'une ressource n'a pas été instanciée correctement lors d'un cycle précédent et est donc marquée pour être détruite et recrée au prochain apply.

```
  # azurerm_mssql_server.server is tainted, so must be replaced
```

Malgré cette opération, la même erreur 404 a persisté.

```
  Resource Group Name: "caca-resources"
  Server Name: "caca-sql-4dv5mj5ol1"): performing CreateOrUpdate: unexpected status 404 (404 Not Found)
with error: ParentResourceNotFound: Failed to perform 'write' on resource(s) of type 'servers/connection
Policies', because the parent resource '/subscriptions/87b59e8c-1393-4d4a-b46c-8b645a549d04/resourceGrou
ps/caca-resources/providers/Microsoft.Sql/servers/caca-sql-4dv5mj5ol1' could not be found.

    with azurerm_mssql_server.server,
    on main.tf line 252, in resource "azurerm_mssql_server" "server":
   252: resource "azurerm_mssql_server" "server" {
```

Soupçonnant un nouveau problème de timing, puisqu'une donnée bien présente dans le *main.tf* ne peut pas être trouvée, je tente de ne lancer que l'apply du serveur avec la commande terraform apply -target=azurerm_mssql_server.server. Malheureusement, l'erreur persiste.

Suppression de la base de données

Il est donc décidé de mettre de côté la base de données en commentant son instanciation afin de conserver une API fonctionnelle.

Erreur lors du terraform destroy

Un terraform destroy est tenté. Il est impossible de récupérer les container lors de cette tentative.

```
  Error: retrieving Container "images" (Account "Account \"cacastorage4dv5mj5ol1\" (IsEdgeZone
 false / ZoneName \"\" / Subdomain Type \"blob\" / DomainSuffix \"core.windows.net\")"): execu
ting request: unexpected status 403 (403 This request is not authorized to perform this operat
ion.) with AuthorizationFailure: This request is not authorized to perform this operation.
  RequestId:5b732086-401e-004d-34cc-b27d3e000000
  Time:2026-03-13T09:32:30.6667531Z

    with azurerm_storage_container.images,
    on main.tf line 207, in resource "azurerm_storage_container" "images":
   207: resource "azurerm_storage_container" "images" {



  Error: retrieving Container "logs" (Account "Account \"cacastorage4dv5mj5ol1\" (IsEdgeZone f
alse / ZoneName \"\" / Subdomain Type \"blob\" / DomainSuffix \"core.windows.net\")"): executi
ng request: unexpected status 403 (403 This request is not authorized to perform this operatio
n.) with AuthorizationFailure: This request is not authorized to perform this operation.
  RequestId:2363f430-001e-00a8-15cc-b22c7c000000
  Time:2026-03-13T09:32:30.7035890Z

    with azurerm_storage_container.logs,
    on main.tf line 214, in resource "azurerm_storage_container" "logs":
   214: resource "azurerm_storage_container" "logs" {
```

Les étapes de débogages sont multiples :

- **Vérification de l'IP stockée dans le *.env* :** avec la commande curl https://api.ipify.org, et après comparaison, l'IP stockée dans le .env n'était plus la bonne, et comme j'ai restreint l'accès à

certains éléments à un IP très précis, cela peut poser des soucis de connexions. J'en profite pour commenter la ligne VM_IP, qui n'est pas utile. Après correction, le problème persiste.

- **Vérification des ports de la machine :** nmap testadmin@20.19.250.243 ne montre aucun port.

```
000 tp_4_cloud_computing (main) $ nmap testadmin@20.19.250.243
Starting Nmap 7.92 ( https://nmap.org ) at 2026-03-13 11:05 CET
Failed to resolve "testadmin@20.19.250.243".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.04 seconds
```

- **Tentative de connexion SSH direct :** la connexion s'arrête à debug3: set_sock_tos: set socket 3 IP_TOS 0x48, indiquant un problème de sockets. Probablement liée au fait qu'Azure utilise le ssh-agent en background mais ne libère jamais les sockets. La commande ssh -i ~/.ssh/key.pub testadmin@4.211.171.82 -vvv -o IPQoS=none -T règle le problème, mais la connexion ne se fait toujours pas correctement après avoir accepté la connexion.
- **Tentative de libérer les sockets en resettant le SSH :** eval $(ssh-agent -s) && ssh-add ~/.ssh/key. L'erreur persiste.
- **Destruction via l'interface Azure du groupe de ressource :** afin de tout reconstruire, le groupe de ressource est effacé. Cela n'a pas réglé les problèmes.
- **Révision de la configuration :** L'action par défaut des conteneurs en « Allow » dans la config.

Après ces étapes, l'erreur n'apparaît plus, mais une autre erreur bloque le déploiement.

Erreur à la récupération de l'IP de la VM

```
TASK [Gathering Facts] ***********************************************************************
[ERROR]: Task failed: Data could not be sent to remote host "{{ lookup('env', 'VM_IP') }}". Ma
ke sure this host can be reached over ssh: hostname contains invalid characters
fatal: [azure_vm]: UNREACHABLE! => {"changed": false, "msg": "Task failed: Data could not be s
ent to remote host \"{{ lookup('env', 'VM_IP') }}\". Make sure this host can be reached over s
sh: hostname contains invalid characters", "unreachable": true}

PLAY RECAP ***********************************************************************************
azure_vm                   : ok=0    changed=0    unreachable=1    failed=0    skipped=0    re
scued=0    ignored=0
```

S'est produite après avoir commenté la ligne VM_IP, commentée parce qu'on ne peut pas connaître le nom de la VM en avance. Hors, elle était utilisée dans *group_vars/api_server.yml*. L'erreur est résolue en changeant le lookup pour récupérer l'output *ip_publique*.

Erreur de connexion SSH à la connexion VM

Une autre erreur bloque le déploiement. Elle indique que la connexion SSH ne peut pas se faire.

```
TASK [Gathering Facts] ***********************************************************
[ERROR]: Task failed: Data could not be sent to remote host "4.211.171.82". Make sure this hos
t can be reached over ssh: Connection timed out during banner exchange
Connection to 4.211.171.82 port 22 timed out

fatal: [azure_vm]: UNREACHABLE! => {"changed": false, "msg": "Task failed: Data could not be s
ent to remote host \"4.211.171.82\". Make sure this host can be reached over ssh: Connection t
imed out during banner exchange\r\nConnection to 4.211.171.82 port 22 timed out", "unreachable
": true}

PLAY RECAP ***********************************************************************
azure_vm                   : ok=0    changed=0    unreachable=1    failed=0    skipped=0    re
scued=0    ignored=0
```

Effectivement, la connexion SSH directement avec la VM échoue.

```
debug1: Local version string SSH-2.0-OpenSSH_10.0
kex_exchange_identification: Connection closed by remote host
Connection closed by 4.211.171.82 port 22
```

Après vérification, toujours aucun port ouvert sur la VM.

```
000 tp_4_cloud_computing (main) $ nmap testadmin@4.211.171.82
Starting Nmap 7.92 ( https://nmap.org ) at 2026-03-13 11:48 CET
Failed to resolve "testadmin@4.211.171.82".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.06 seconds
```

Pourtant, dans les Network Settings de la VM dans l'interface Azure, cela indique que le port 22 est bien ouvert.

Priority ↑	Name	Port	Protocol	Source	Destination	Action
Inbound port rules (5)						
100	allow-ssh	22	TCP	176.183.133.180/32	Any	Allow
110	allow-api	8000	TCP	176.183.133.180/32	Any	Allow

Une autre vérification de l'adresse IP locale s'impose : avec curl https://api.ipify.org, elle avait encore changé, le *.env* est donc corrigé. Le script de déploiement est relancé.

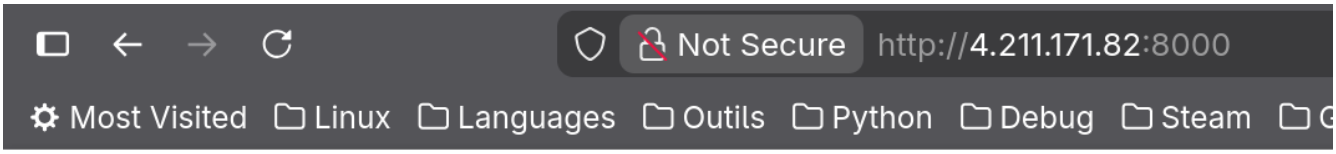
Conclusion

Après cette dernière tentative de déploiement, aucune erreur n'est détectée.

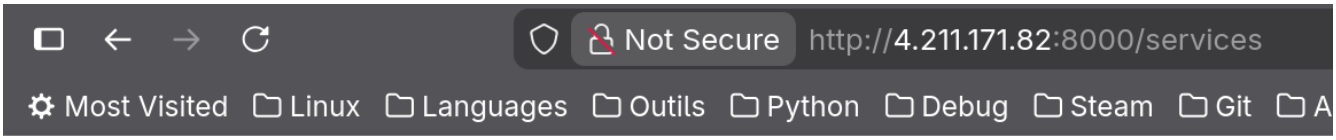
```
PLAY RECAP *********************************************************************************
azure_vm                   : ok=14   changed=11   unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

API accessible sur : http://4.211.171.82:8000
```

L'API est désormais fonctionnelle, accessible à l'adresse et au port indiqué.



Hello World



Je suis une page qui présente les services